Kerbo-S Sprint 1 Reflection Report

AI-Assisted Kerberos Security Auditor | CSCE 4907 Senior Capstone

GitHub URL: https://github.com/chancellorr/Cybersecurity-Capstone.git

Team: Chance Edwards, Ben Grider, David Pantoja, Jessica Yescas | Revision date: October 29, 2026

Accomplishments

Our team (Chance Edwards, Ben Grider, David Pantoja, Jessica Yescas) did not meet the Sprint 1 requirements on time: there was no repository, no design document and an incomplete reflection. We then recovered, and the work below is what now exists for Kerbo-S, our AI-assisted Kerberos security auditor.

Documents (all revised or written after the instructor's ten feedback items on the proposal)

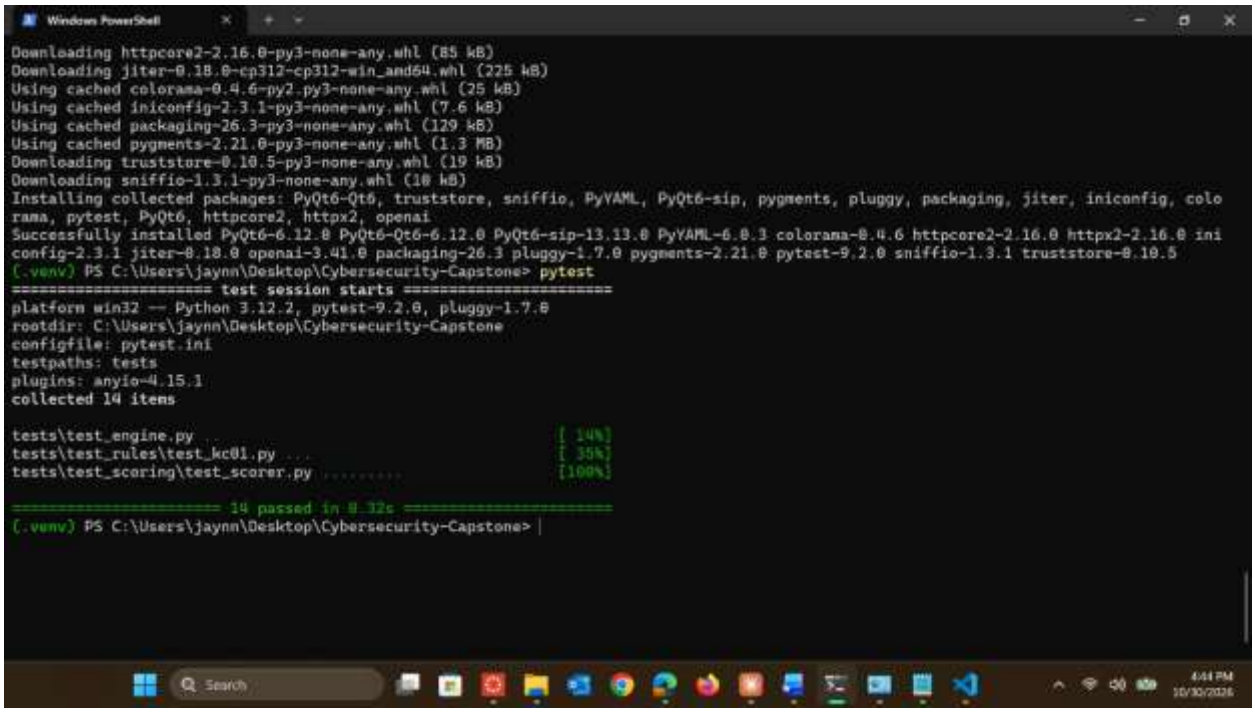
- **Requirements Document v2.0.** Rewritten with an 18-item change table (C-01 to C-18), a Section 1.2 that maps each feedback item to the requirement that answers it, and a New / Changed / Retained tag on every requirement. It now defines concrete rule checks KC-01 to KC-11, a weighted risk score (Impact 40%, Exploitability 30%, Exposure 20%, Compliance 10%) with severity bands (Low 0-24, Medium 25-49, High 50-74, Critical 75-100), a threat model, an MVP boundary and measurable success metrics.
- **Design Document v2.0.** Describes the pipeline from collection on the KDC to analysis on a workstation, with a snapshot boundary between them. It also covers the AI Gateway (sanitize, pseudonymize, schema validation, mock provider), the SQLite schema with immutable findings, security controls and design trade-offs, and records seven design changes (D-01 to D-07).
- **Change Document.** Lists every change from the proposal: 10 requirement, 7 design, 2 implementation and 5 process changes.
- **Draft Test Plan.** 57 test cases across environment, collection, rules, scoring, database, AI, reports, UI, security and AI evaluation, tied to 15 test scenarios (TV-01 to TV-15). All are currently marked Not Yet Tested, which is accurate because Sprint 2 is when they run.

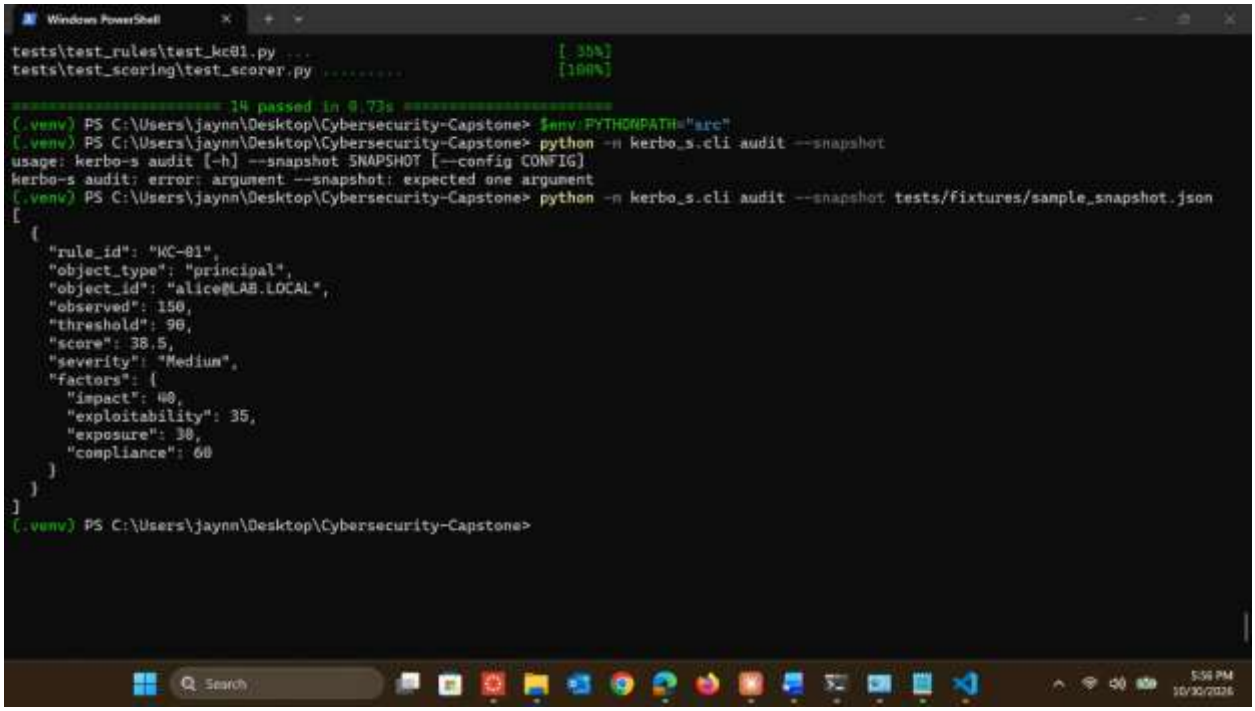
Repository and working code

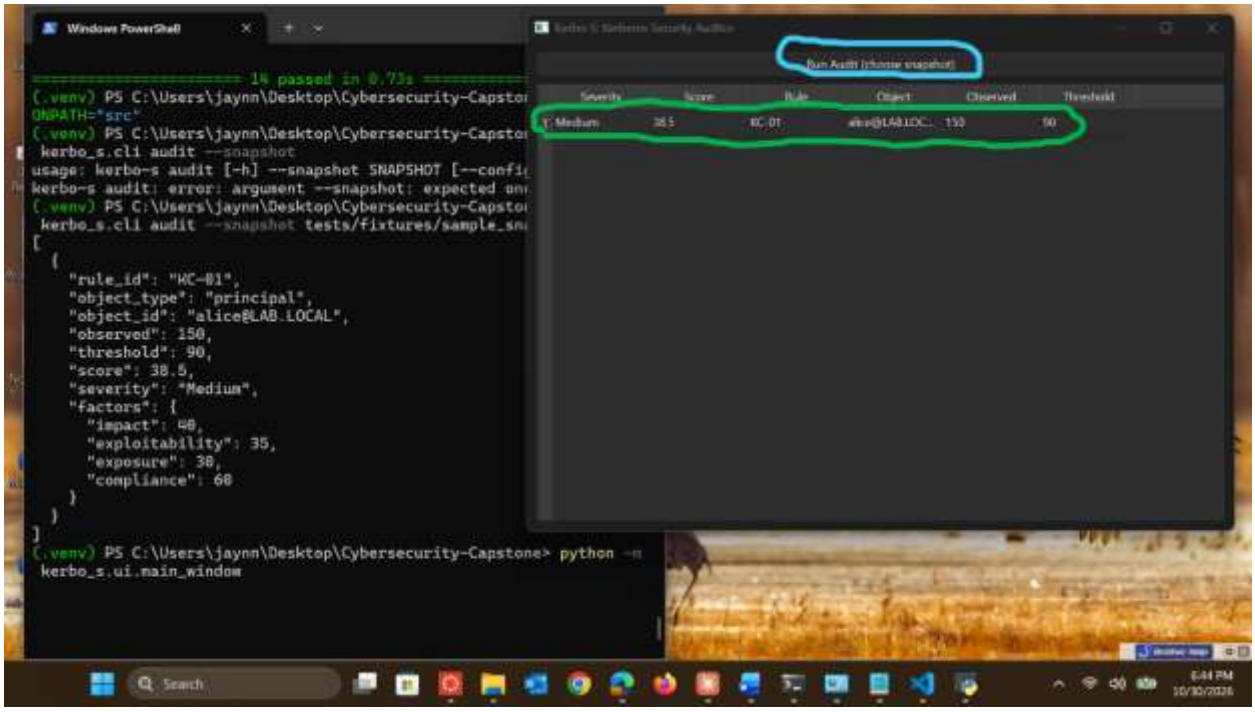
- The GitHub repository (chancellorr/Cybersecurity-Capstone) was made **public** on Nov 1, so the instructor and any TA can open it without an invitation. Before that, every commit was searched for passwords, keys and tokens; none were found. It holds a README with project status and quick

start, a .gitignore that keeps keys, databases and real snapshots out of Git, a requirements file, the recorded test runs in docs/, and the five Sprint 1 documents as PDFs in docs/sprint1/.

- A small walking skeleton runs end to end: load a snapshot, apply rule KC-01 (flags user passwords older than 90 days), score the finding, and show it in a command-line report or a PyQt6 window with a findings table. It is intentionally small. It proves the structure works and gives every teammate the same starting point.
- The skeleton has 14 automated tests, and all 14 pass. They cover the severity bands, the baseline scores for KC-01, KC-02 and KC-07, KC-01 boundary cases (91 days flagged, exactly 90 days not flagged, service principals ignored), and a check that the same snapshot always gives the same result.







Process

- **Trello board.** Every card is tied to a requirement or test case and a team slice. Since Nov 1, every Sprint 2 card also names an owner, a due date and a "Done when" list, and a Blocked list shows work that is waiting on something.

- **Git workflow.** Three of four teammates have now completed the full branch, commit, push, pull request, review and merge cycle: Jessica (PR #1, Oct 30), Chance (PR #2) and David (PR #3), both merged Nov 2. A team Git and Setup Guide is published, and each teammate has a step-by-step Sprint 2 guide with their tasks in order.

Use of AI

We used Claude (Anthropic) as a drafting and coding assistant during Sprint 1. We did not use AI inside the Kerbo-S product itself yet. The AI Gateway is designed, and its mock provider is a Sprint 2 task.

Use	What Claude did	How we checked it
Choosing an approach	Compared options and recommended the architecture: a deterministic rule engine decides and scores, and AI only explains findings	Matched against the instructor's feedback item on separating the AI from the audit logic
Use	What Claude did	How we checked it
Requirements v2.0	Applied the ten feedback items, drafted the change table and rule definitions	Read each requirement against the feedback list and the original proposal; kept our own wording where we wanted it
Design, change and test plan documents	Drafted from the revised requirements	Cross-checked IDs (rule IDs, TV and TC numbers, requirement tags) between the four documents
Starter code and tests	Wrote the skeleton, KC-01, the scorer and 14 tests	Ran it ourselves: pytest passed 14 of 14, the command-line audit printed the expected KC-01 finding for alice, and the window opened and showed it

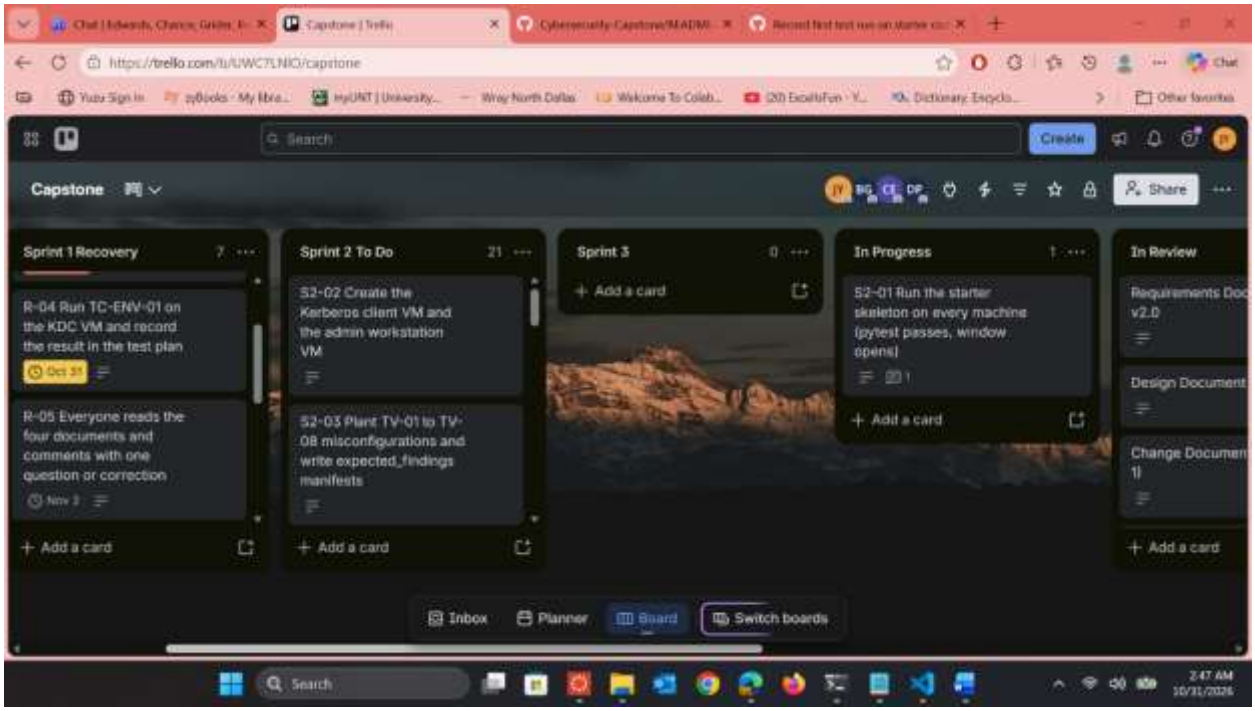
Git and GitHub guidance	Walked one teammate through clone, branch, push and pull request	Followed the steps on a real machine; every error was fixed and recorded (see below)
Trello	Created the 41 cards through the Trello connector	Reviewed the board against the plan

Where we did not simply trust the AI. The first pytest and CLI runs failed (missing PyYAML and pytest, caused by installing packages globally instead of in the virtual environment). The AI's first answer for one Git check was also wrong: it said a command should print nothing, and the real output showed the file was not ignored. We caught that by reading the actual output. Running the code on our own machine, rather than accepting a description of it, is the check we rely on.

Limits we know about. The AI-written documents have not yet been read through by every teammate, and nothing has been tested against the real Kerberos lab, only the sample snapshot. Both are Sprint 2 items.

What Worked Well

- **Turning instructor feedback into a checklist.** Mapping each of the ten feedback items to a numbered change (C-01 to C-18) made the rewrite verifiable instead of vague, and gave us a clear answer to "did we fix what was asked?"
- **One ID scheme across every document.** Rule IDs (KC-01 to KC-11), test scenarios (TV-01 to TV-15), test cases (TC-...) and requirement tags let us trace a rule from requirement to design to test.
- **A deterministic core.** Keeping scoring in plain code with fixed weights means the same input always gives the same result. A test (`test_audit_is_deterministic`) proves it. • **A small walking skeleton.** Getting one rule through snapshot, scoring and window early showed the project structure works before we write the other ten rules. • **Running things for real.** The venv and pytest problems were found and fixed because one teammate ran every step on her own machine and reported the exact error. Her results are recorded in docs/test-runs.md.
- **Trello with traceable cards.** Each of the 41 cards names its requirement, test case and team slice (A: lab and collectors, B: audit engine and scoring, C: GUI and storage, D: AI and reports), so ownership is clear.



What Needs Improvement

- **We missed the Sprint 1 deliverables.** At the deadline there was no repository, no design document and an incomplete reflection. The cause was not technical. Work was not divided early, and most of it fell on one person. This is the main thing to fix.
- **Uneven participation.** The proposal and the recovery work were done mostly by one teammate. The other three teammates need to show up in the repository history and on Trello, with real commits and cards moved by their owners.
- **The skeleton has run on three of four machines.** Trello card S2-01 (run the starter on your own machine) was due Nov 1. Jessica, Chance and David have each run pytest (14 passed), the CLI audit and the window on their own Windows machines and recorded it in docs/. Ben has not yet.
- **Setup friction.** We lost time to Git sign-in problems (an Administrator PowerShell with no GitHub login), starter files copied to the wrong folder, a package installed globally instead of in the virtual environment, and typos in commands. A written team guide is being prepared so no one repeats these.
- **Repository cleanup.** Access is solved: the repository is public. Still open (card R-02): tidy the .gitignore and delete the old feature-1 branch, which is fully merged.
- **Documents not yet reviewed by everyone.** All four documents were drafted with AI help, and every teammate was asked to read them and leave at least one comment (R-

05, due Nov 2). None have been left yet. Part of the cause was technical: the documents were shared with Commenter access, which did not allow comments; this was found and fixed on Nov 2 by switching to Editor access.

- **No real-environment testing.** Everything so far runs on a sample snapshot. We have not yet captured real `kadmin.l` lanted the misconfigurations in the lab (TV-01 to TV-08).
- **No automated checks on pull requests.** The first pull request showed Checks 0, so tests run only when someone remembers to run them.

Team participation record (Sprint 2 start, Nov 1 to 2, 2026)

S2-01 (run the starter skeleton on your own machine and open one pull request) was due **Sun Nov 1, 6 PM**. At that deadline, 1 of 4 members had done it (Jessica, PR #1, Oct 30). Status as of **Mon Nov 2, 6:40 PM Central**, from the GitHub repository and the Trello board:

Person	What happened (Mon Nov 2)	Missing
Chance	3:45 PM: pushed his test run, **PR #2**. 14 tests pass.	PR link not posted on Trello. R-02 not finished (`.gitignore` unchanged, `feature-1` branch not deleted; due 12 PM). R-07 (export documents into the repo) due 6 PM, not done.
David	6:04 PM: pushed his test run, **PR #3**, posted the link on Trello at 6:21 PM, requested review from Ben. 14 tests pass.	Monday models PR (S2-06, step 1) not started.
Ben	Nothing: no commits, no pull requests, no Trello activity.	S2-01; R-05; **S2-00 (build the KDC VM), due 6 PM**.

- **S2-01** now stands at 3 of 4 members (Ben missing).
- **R-05** (each member reads the four Sprint 1 documents and leaves at least one comment) was due Mon Nov 2, 12 PM. As of 6:40 PM there were **no comments** in any of the four documents. Cause found at about 8 PM, testing live with David by phone: the documents had been shared with Commenter access, which did not show the comment option; only Editor access did. Teammates could not comment until their access was changed to Editor on Mon Nov 2, about 8 PM. David's test comment (7:48 PM) confirmed it works.

- PRs #2 and #3 were checked: each adds one documentation file, merges cleanly, and keeps all 14 tests passing. Jessica reviewed and merged both on Mon Nov 2 (by 7:16 PM) and deleted their branches; main passes 14 tests.
- **Risk:** the KDC lab VM (Ben, S2-00) is the dependency for the real test data, the parser and end-to-end tests, and the Friday test run. The proposed fallback point is Tue Nov 3, 12 PM: if the VM is not up, S2-00 moves to Blocked and the team reassigns the lab build.

Update, Tue Nov 3, 10:58 AM Central (GitHub repository, Trello board and the four Sprint 1 documents):

Person	What happened since Mon 6:40 PM	Missing
Chance	No new commits or Trello activity after PR #2 was merged.	R-02 still open (`.gitignore` unchanged, `feature-1` still exists; due Mon 12 PM). R-07 not done (no `docs/sprint1/` ; due Mon 6 PM). No comment in the four documents.
David	Mon 6:50 PM: commented "Done David" on R-05 in Trello. Mon about 8 PM: tested comments with Jessica by phone; his test comment on his guide worked.	No comment found in the four Sprint 1 documents. Models PR (S2-06, step 1) not opened.
Ben	Nothing: no commits, no pull requests, no Trello activity, no document comments.	S2-01 (due Sun Nov 1), R-05 (due Mon 12 PM), S2-00 build the KDC VM (due Mon 6 PM), R-04 run TC-ENV-01 (due Tue 12 PM).
Person	What happened since Mon 6:40 PM	Missing
Jessica	Mon 1:30 AM: shared the documents and guides, posted the plan in Teams. 7:16 PM: reviewed and merged PRs #2 and #3. About 8 PM: found and fixed the document access problem. 8:15 PM: sent David the model code he needs. Tue: finalized this report and the resubmission.	None for Sprint 1.

- Because Ben has not started the KDC lab, the Sprint 1 test case TC-ENV-01 remains Not Yet Tested in the test plan.

Focus for Sprint 2

Sprint 2 is due **Sat Nov 7**, one week after this recovery. The earlier plan assumed four weeks, so it was cut to what four people can build, test and demonstrate in one week: a real MIT Kerberos lab, real collected data, rules KC-01 to KC-09, findings stored in SQLite that cannot be edited, a findings table with filters and clear error messages, and test results backed by evidence. AI explanations, PDF reports, the log-based rules (KC-05, KC10, KC-11), snapshot hashing and the evaluation script move to Sprint 3. Code freezes **Thu Nov 5 at 6 PM**.

Day	Main work	Owner	Test cases
Mon Nov 2	Everyone runs the starter (S201); data models extended for policies and configuration	All; David	None
Tue Nov 3	Build the KDC lab VM and run TC-ENV-01; resubmit Sprint 1	Ben; Jessica	TC-ENV-01
Wed Nov 4	Capture real lab data and plant the test scenarios with expected-findings files written first; parsers; rules KC-02 to KC-04 and KC-06 to KC-09	Ben; David; Chance; Jessica	TC-ENV-02, TC-COL-01 to 03, TC-KC-02 to 09
Day	Main work	Owner	Test cases
Thu Nov 5	SQLite storage; findings table with filters; error messages; end-to-end tests on the clean and planted lab data. Code freeze 6 PM	David; Chance	TC-DB-01, TC-DB-02, TCUI-02, TC-UI-06, TC-KC-12, TC-KC-15

Fri Nov 6	Run every Sprint 2 test and record evidence; update the documents; rehearse the demo	Ben; Jessica; all	All of the above
Sat Nov 7	Submit Sprint 2	Jessica	None

Status of the Sprint 1 recovery items (Tue Nov 3)

- **Resubmit the Sprint 1 package** (the four documents, this report and the repository link): done Nov 3.
- **Instructor access to the repository:** done. The repository is public as of Nov 1.
- **Each teammate makes one commit and opens a pull request:** 3 of 4 done (Jessica, Chance, David). Ben outstanding.
- **A named owner on every Trello card:** done Oct 31.
- **Confirm whether Ubuntu-only support for the lab is acceptable:** asked in this resubmission.
- **Run TC-ENV-01:** not done; depends on the KDC lab VM (Ben, S2-00).

How we will know it worked. By the end of Sprint 2, `pytest` passes on every teammate's machine, the audit runs against real lab output, findings are saved to SQLite and cannot be edited, and the test plan shows real Pass or Fail results instead of Not Yet Tested.